

ConnectedDesk Technology Stack & Documentation

Overview

ConnectedDesk is a comprehensive collaborative workspace designed to streamline team communication and project management. It integrates real-time chat, task tracking, collaborative tools, and data visualization into a single unified platform.

⚙️ Technology Stack

Frontend (Client)

The frontend is built as a Single Page Application (SPA) using modern React and Vite.

- **React 19:** The core library for building the user interface.
- **Vite:** A fast build tool that provides a near-instantaneous development environment.
- **Context API:** Used for global state management (Authentication and Theme).
- **React Router DOM:** Handles client-side routing and navigation.
- **Framer Motion:** Powering smooth, high-quality animations throughout the UI.
- **Recharts:** Used for rendering dynamic charts and data visualizations on the dashboard.
- **@hello-pangea/dnd:** Enables drag-and-drop functionality for the Kanban-style task management.
- **Socket.io-client:** Facilitates real-time, bidirectional communication with the server.
- **Simple-Peer:** Implements WebRTC for peer-to-peer video calling.
- **React Toastify:** Provides elegant popup notifications for user feedback.

Backend (Server)

The backend is a robust RESTful API and WebSocket server powered by Node.js.

- **Node.js:** The cross-platform JavaScript runtime.
 - **Express.js:** A minimal and flexible web application framework for handling API routes.
 - **MongoDB:** A NoSQL database used for flexible and scalable data storage.
 - **Mongoose:** An ODM (Object Data Modeling) library for MongoDB and Node.js.
 - **Socket.io:** Powers real-time features like chat messaging, typing indicators, and whiteboard synchronization.
 - **JSON Web Token (JWT):** Used for secure, stateless user authentication.
 - **Bcryptjs:** Handles secure password hashing.
 - **Multer:** Middleware for handling `multipart/form-data`, primarily for file uploads.
 - **CORS:** Configured to handle cross-origin requests securely.
-

How It Works

1. Communication Architecture

The application uses a hybrid communication model:

- **REST API:** Conventional HTTP requests (GET, POST, PUT, DELETE) are used for standard operations like fetching tasks, updating profiles, and authentication.
- **WebSockets (Socket.io):** Used for features requiring "instant" updates. When a user sends a message or draws on the whiteboard, the server broadcasts the event to all relevant clients immediately.

- **WebRTC:** For video calls, Socket.io acts as a "signaling" server to help two users find each other. Once connected, the actual video data flows directly between the users (Peer-to-Peer) for low latency.

2. Core Feature Workflow

Authentication

- Users register/login through the frontend.
- The server validates credentials and sends back a **JWT token**.
- The client stores this token and includes it in the `auth-token` header for all authorized requests.

Real-time Chat

- Messages are saved to MongoDB as they are sent.
- Simultaneously, a `new_message` event is emitted via Socket.io.
- Recipients in the same chat room receive the event and update their UI instantly without refreshing.

Kanban Tasks

- Tasks are categorized into statuses (To Do, In Progress, Done).
- Dragging a task updates its status in the backend via an API call, while the frontend reflects the change instantly using `hello-pangea/dnd`.

Collaborative Whiteboard

- Every "stroke" or action on the whiteboard is shared via Socket.io.
- The server maintains a temporary memory of the whiteboard state so that late-joiners can see the previous drawings.

Dashboard & Analytics

- The dashboard aggregates data from tasks, meetings, and activity logs.
- Recharts processes this data into visual insights, helping users track productivity.

Project Structure

- **/client:** Contains the React application, styles, and assets.
- **/server:** Contains the Express API, database models, and route controllers.
- **/server/uploads:** Stores uploaded files (profile pictures, resources).

Getting Started

1. **Database:** Ensure a MongoDB instance is running.
2. **Environment:** Set up `.env` files in both `client/` and `server/` with appropriate keys (DB URI, JWT Secret, API URLs).
3. **Execution:**
 - Server: `cd server && npm run dev`
 - Client: `cd client && npm run dev`